

PRD — Smart Temporary Address Management (Delivery Apps)

Project Name: Smart Temporary Address Management System (Delivery Apps)

Role: Product Manager (Self-Initiated Project)

Prepared by: Mani Rachamallu

Duration: June 2026

Status: Final

Last updated: 2026-09-03

1. Executive Summary

Frequent movers - students in hostels, professionals who alternate between home and office, and travellers staying in hotels—face repeated friction managing delivery addresses in delivery apps. Today, users must manually add and delete short-term addresses, and at checkout they often see a long list of outdated entries. Under time pressure, users can accidentally select the wrong saved address, leading to failed deliveries, refunds/support overhead, and a degraded customer experience.

The Smart Temporary Address Management System introduces **Temporary Stay** addresses with user-selected expiry. When saving an address, users can mark it as temporary and choose an **expiry date/time** (with quick presets). Before expiry, the app can notify users and let them take immediate actions: extend the stay, convert the address to permanent, or archive it. Once expired, the address is **auto-archived** (not deleted) to keep checkout clean, while still allowing one-tap restoration for repeat stays.

Expected impact: fewer wrong-address orders, faster checkout, reduced support tickets related to address issues, and improved repeat ordering for frequent movers.

2. Problem Statement

People who frequently move between hostels, offices, hotels, relatives, homes, and temporary residences must repeatedly add and remove delivery addresses in delivery applications.

This leads to:

- Wrong deliveries due to outdated address selection
 - Checkout friction (extra steps, higher hesitation)
 - Increased cognitive load (remembering what to remove)
 - Reduced ordering frequency (users avoid ordering when unsure)
-

3. User Research

~ Initial discovery assumptions

3.1 Personas

Persona 1: Rahul (22)

- Student, lives in hostel
- Pain points:
 - Forgets to remove old addresses
 - Likely to pick the wrong address during fast checkout

Persona 2: Priya (28)

- Software Engineer, travels frequently
- Pain points:
 - Orders to hotels, offices, temporary rentals
 - Needs addresses only for short periods
 - Wants “set once, auto-cleanup” behaviour

3.2 Key insights

- Address management is a “background task” users want to minimise.
 - Checkout is time-sensitive; address mistakes are costly.
 - Users prefer “keep history but don’t clutter checkout.”
-

4. Goals & Non-Goals

4.1 Goals

- Reduce address-management effort for frequent movers.
- Reduce wrong-address deliveries (and related support tickets).
- Reduce time spent on address selection at checkout.

4.2 Non-goals (v1)

- Changing delivery partner operational flows (rider navigation, routing logic).
 - Address validation / geocoding improvements beyond current system.
 - Multi-user household address sharing.
-

5. Scope & User Journey

5.1 Current journey (today)

Open app → Profile/Checkout → Manage addresses → Add/Edit address → Checkout → (Often forget to remove old address)

5.2 Proposed journey (with this feature)

1. User adds an address → selects **Permanent** or **Temporary Stay**
 2. If Temporary Stay → user selects **expiry date/time**
 3. Checkout shows clean address list with badges for temporary + expiry hint
 4. On expiry → address auto-moves to **Archived**
 5. User can restore an archived address (restores as temporary for 3 days)
-

6. Product Requirements Document (PRD)

6.1 Objective

Reduce address-management effort and wrong-address deliveries for frequent movers.

6.2 Success metrics

Primary

- Wrong-address order rate (delivery failure/reroute/refund due to address) ↓
- Checkout completion rate ↑
- Time spent in address selection step ↓

Secondary

- Support tickets tagged "wrong address / address change" ↓
- Orders per active user for frequent movers segment ↑
- Median address list size shown at checkout ↓

6.3 User stories

- As a frequent traveler, I want an address to expire automatically so my checkout stays clean.
 - As a user ordering from a hotel, I want to set a specific end date/time so I don't forget to remove the hotel address.
 - As a user returning to a previous place, I want to restore an old address quickly.
-

7. Functional Requirements

FR1 — Address type selection

- In Add/Edit Address, user can select:
 - **Permanent** ("Always saved")

- **Temporary Stay** ("Auto-expires")

FR2 — Expiry date/time picker (Decision locked)

- If **Temporary Stay** is selected, user must **choose an expiry date/time** before saving.
- Provide quick presets (example): **12 hours, 1 day, 3 days, 1 week**, plus **Custom**.
- Validation:
 - Expiry must be in the future.
 - Save is disabled until expiry is selected.

FR3 — Visibility in checkout

- Temporary addresses must show a **TEMPORARY** badge and "Expires in X" hint in address lists.
- Expired addresses must not be selectable at checkout.
- At "Place Order", server must validate the selected address is active; if expired, prompt user to select another address.

FR4 — Auto-archive on expiry

- On expiry, temporary addresses are **auto-archived** (not deleted).
- Archived addresses remain available for restoration and conversion.

FR5 — Archived address actions

- In Archived Addresses, user can:
 - **Restore**
 - **Make Permanent**
 - **Delete** (manual only)

FR6 — Restore behaviour (Decision locked)

- Restore always returns the address as **Temporary Stay** with a default expiry of **3 days from restore time**.
- Recommended UX:
 - Toast: "Restored for 3 days" with optional "Change" action.

FR7 — Expiry alerts & actionable notifications

- If enabled, send "Address expiring" notification **12 hours before expiry**.
- Notification CTAs:
 - **Extend Stay**
 - **Make Permanent**
 - **Archive Now**

FR8 — Address settings ("Travel Mode")

- Setting: **Treat new addresses as temporary** (Travel Mode).
- When enabled:
 - Default Address Type = Temporary Stay
 - Default expiry preset highlighted = **3 days** (user can still choose any expiry)

FR9 — Smart Suggestions (typing assist)

- Optional toggle: detect likely hotels/hostels/Airbnbs while typing and suggest Temporary Stay / Travel Mode.
-

8. Non-Functional Requirements

- Checkout/address list load time: **< 2 seconds**
- Reliability: **99.9% uptime** for address services
- Privacy & security:
 - Addresses treated as sensitive personal data
 - Secure at rest and in transit

- Access control enforced server-side
 - Consistency:
 - Expiry logic is server-side source of truth (works across devices)
 - Timezones:
 - Store expiry in UTC; display in user's local timezone
-

9. UX Notes (from prototype)

- Saved Addresses hub includes:
 - Active Saved (permanent)
 - Temporary Stays (shows "Expires in X")
 - Archived (expired history, retained for 30 days)
 - Archived page explains: "Nothing is ever deleted automatically."
 - Notifications include clear actions: Extend, Make Permanent, Archive.
-

10. Edge Cases & Guardrails

- If user has no active addresses, block checkout and require adding a new address.
 - If an address expires during a checkout session:
 - Revalidate at "Place Order" and force reselection.
 - Offline mode:
 - Expiry still enforced server-side; UI reconciles at next sync.
 - Ongoing orders:
 - An order must remain linked to the address snapshot used at time of ordering.
-

11. Feature Prioritization (RICE — initial)

1. Checkout filtering/validation for expired addresses (highest risk reduction)
2. Temporary address + expiry picker
3. Auto-archive + archived management actions
4. Expiry notifications with CTAs
5. Smart suggestions + travel mode enhancements

Confidence: Initial prioritization is assumption-based and should be validated through user research, baseline product data, and usability testing.

12. Technical Architecture (High-level)

Mobile UI



Address Service API



Address Database



Scheduler/Jobs (expiry transitions, reminders)



Notification Engine (push)

13. Data Model / SQL Design (conceptual)

13.1 Tables

users

- user_id (PK)
- name
- phone/email
- created_at

addresses

- address_id (PK)
- user_id (FK)
- label (Home/Office/Hotel/Friend/Custom)
- line1, line2, city, state, pincode, country
- lat, lng (optional)
- address_type (PERMANENT | TEMPORARY)
- expires_at (nullable for permanent; required for temporary)
- status (ACTIVE | ARCHIVED) (*expiry is derived from expires_at; archive is explicit*)
- notifications_enabled (boolean)
- created_at, updated_at
- last_used_at (optional)

orders

- order_id (PK)
- user_id (FK)
- address_id (FK)
- address_snapshot (JSON/text — recommended to avoid future edits affecting history)
- order_status
- created_at

13.2 Key queries

- Active addresses for checkout:
 - `WHERE user_id = ? AND status='ACTIVE' AND (expires_at IS NULL OR expires_at > NOW())`
- Expiry job (archive on expiry):

- `UPDATE addresses SET status='ARCHIVED' WHERE status='ACTIVE' AND address_type='TEMPORARY' AND expires_at <= NOW()`
-

14. Open Questions

- **Reminder Strategy**

Is a single 12-hour reminder sufficient, or should users receive additional reminders closer to expiry?

- **Address Expiry During an Active Order**

If an address expires while an order is being prepared or delivered, should the address remain temporarily active until the order is completed?

- **Notification Preferences**

Should users be able to disable expiry reminders or customise notification preferences?

- **Future Personalization**

Should the system eventually suggest temporary expiry durations based on the user's previous address usage patterns?